CSCI3230 / ESTR3108 2026–27 First Term Assignment 2

I declare that the assignment here submitted is original except for source material explicitly acknowledged, and that the same or closely related material has not been previously submitted for another course. I also acknowledge that I am aware of University policy and regulations on honesty in academic work, and of the disciplinary guidelines and procedures applicable to breaches of such policy and regulations, as contained in the following websites.

University Guideline on Academic Honesty:
`http://www.cuhk.edu.hk/policy/academichonesty/`
Faculty of Engineering Guidelines to Academic Honesty:
`http://www.erg.cuhk.edu.hk/erg-intra/upload/documents/ENGG_Discipline.pdf`

Student Name: Jiang Huanyun
Student ID: 1155233165

Note. All computations were performed in Python (NumPy / scikit-learn); the hand calculations of Questions 1, 3 and 4 were independently cross-checked against the numerical results.

Question 1: SVM optimization and prediction

Data. The training set contains $m = 8$ points $\{(\mathbf{x}_i, y_i)\}_{i=1}^{8}$ and the test set contains 8 points:

	Training set								Test set							
i	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
x_1	3.5	5	3	5.5	4.5	3.5	3	4	5.5	3.5	4.5	2.5	3.5	2.5	4	5
x_2	−2	1.5	3	−1	−5	−6.5	−4	−3.5	0.5	−0.5	1.5	−1	−4.5	−4	−6	−5
y	1	1	1	1	−1	−1	−1	−1	1	1	1	1	−1	−1	−1	−1

(a) Dual problem, support vectors, optimal hyperplane

Primal problem. For a hard-margin SVM we solve

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 \qquad \text{s.t.} \qquad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \ldots, 8.$$

Dual problem. Introducing Lagrange multipliers $\alpha_i \geq 0$ for the constraints, the Lagrangian is

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{8} \alpha_i \big[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1\big].$$

Setting $\partial\mathcal{L}/\partial\mathbf{w} = 0$ and $\partial\mathcal{L}/\partial b = 0$ gives

$$\mathbf{w} = \sum_{i=1}^{8} \alpha_i y_i \mathbf{x}_i, \qquad \sum_{i=1}^{8} \alpha_i y_i = 0,$$

and substituting back yields the dual problem

$$\boxed{\max_{\boldsymbol{\alpha}} \ \sum_{i=1}^{8} \alpha_i - \frac{1}{2}\sum_{i=1}^{8}\sum_{j=1}^{8} \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^\top \mathbf{x}_j \qquad \text{s.t.} \qquad \sum_{i=1}^{8} \alpha_i y_i = 0, \quad \alpha_i \geq 0 \ \forall i.}$$

Support vectors. Given $\alpha_1 = \alpha_8 = 0.8$ and $\alpha_2 = \cdots = \alpha_7 = 0$, the KKT complementarity condition $\alpha_i\big(y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1\big) = 0$ states that only points with $\alpha_i > 0$ lie on the margin. Hence the support vectors are

$$\boxed{\mathbf{x}_1 = (3.5, -2)^\top \text{ with } y_1 = +1 \qquad \text{and} \qquad \mathbf{x}_8 = (4, -3.5)^\top \text{ with } y_8 = -1.}$$

Optimal hyperplane. First,

$$\mathbf{w}^* = \sum_{i=1}^{8} \alpha_i y_i \mathbf{x}_i = 0.8 \cdot 1 \cdot \begin{pmatrix} 3.5 \\ -2 \end{pmatrix} + 0.8 \cdot (-1) \cdot \begin{pmatrix} 4 \\ -3.5 \end{pmatrix} = \begin{pmatrix} 2.8 \\ -1.6 \end{pmatrix} - \begin{pmatrix} 3.2 \\ -2.8 \end{pmatrix} = \boxed{\begin{pmatrix} -0.4 \\ 1.2 \end{pmatrix}}.$$

For the bias, using each support vector $y_s(\mathbf{w}^\top \mathbf{x}_s + b) = 1$:

$$\begin{aligned} \mathbf{x}_1: \quad & b = y_1 - \mathbf{w}^{*\top} \mathbf{x}_1 = 1 - \big[(-0.4)(3.5) + (1.2)(-2)\big] = 1 - (-1.4 - 2.4) = 4.8, \\ \mathbf{x}_8: \quad & b = y_8 - \mathbf{w}^{*\top} \mathbf{x}_8 = -1 - \big[(-0.4)(4) + (1.2)(-3.5)\big] = -1 - (-1.6 - 4.2) = 4.8. \end{aligned}$$

Both give $b^* = 4.8$ (and averaging the two gives the same value). The optimal hyperplane (function) is therefore

$$\boxed{f(\mathbf{x}) = \mathbf{w}^{*\top} \mathbf{x} + b^* = -0.4\, x_1 + 1.2\, x_2 + 4.8 = 0}$$

i.e. $x_2 = \frac{1}{3} x_1 - 4$, with margin $\frac{2}{\|\mathbf{w}^*\|} = \frac{2}{\sqrt{1.6}} \approx 1.581$.

(b) Predicting the test set

Given the optimal hyperplane, the prediction rule is the sign of the decision function:

$$\hat{y} = \operatorname{sign}\big(f(\mathbf{x})\big) = \operatorname{sign}\big(\mathbf{w}^{*\top} \mathbf{x} + b^*\big) = \operatorname{sign}\big(-0.4\, x_1 + 1.2\, x_2 + 4.8\big),$$

where $\hat{y} = +1$ means class $+1$ and $\hat{y} = -1$ means class -1. Evaluating:

Test point	(x_1, x_2)	$-0.4x_1 + 1.2x_2 + 4.8$	$\hat{y}$	y	correct?
1	$(5.5, 0.5)$	$-2.2 + 0.6 + 4.8 = 3.2$	$+1$	$+1$	yes
2	$(3.5, -0.5)$	$-1.4 - 0.6 + 4.8 = 2.8$	$+1$	$+1$	yes
3	$(4.5, 1.5)$	$-1.8 + 1.8 + 4.8 = 4.8$	$+1$	$+1$	yes
4	$(2.5, -1)$	$-1.0 - 1.2 + 4.8 = 2.6$	$+1$	$+1$	yes
5	$(3.5, -4.5)$	$-1.4 - 5.4 + 4.8 = -2.0$	-1	-1	yes
6	$(2.5, -4)$	$-1.0 - 4.8 + 4.8 = -1.0$	-1	-1	yes
7	$(4, -6)$	$-1.6 - 7.2 + 4.8 = -4.0$	-1	-1	yes
8	$(5, -5)$	$-2.0 - 6.0 + 4.8 = -3.2$	-1	-1	yes

So the predictions are $\boxed{+1,\ +1,\ +1,\ +1,\ -1,\ -1,\ -1,\ -1}$, i.e. test points 1–4 are classified as class $+1$ and points 5–8 as class -1, giving 100% test accuracy.

(c) Removing training points

- **Remove the 2nd training point** $(5, 1.5, +1)$. This point has $\alpha_2 = 0$, i.e. it is not a support vector and does not lie on the margin. Removing a non-support vector does not change the optimal solution: the remaining dual problem (with the same $\alpha_1 = \alpha_8 = 0.8$) still yields $\mathbf{w}^* = (-0.4, 1.2)^\top$, $b^* = 4.8$, hence **the test predictions do not change**.

- **Remove the 8th training point** $(4, -3.5, -1)$. This point is a support vector ($\alpha_8 = 0.8 > 0$), so the margin hyperplane must change. Re-solving the SVM on the remaining 7 points (numerically, exact solution) gives a different classifier $\mathbf{w}^* \approx (0.2353, 0.9412)^\top$, $b^* \approx 2.0588$. *However*, the decision boundary still separates the two classes in exactly the same way for the test points: all eight test points are still classified correctly, i.e. **the test predictions also do not change** (100% accuracy is retained). This is because the margin is supported by the point $\mathbf{x}_1 = (3.5, -2, +1)$ and the new closest negative point $\mathbf{x}_5 = (4.5, -5, -1)$, which yields a classifier whose decision region covers all test points as before.

- *Numerical verification:* removing point 2 gives exactly $\mathbf{w}^* = (-0.4, 1.2)^\top$, $b^* = 4.8$; removing point 8 gives $\mathbf{w}^* = (0.2353, 0.9412)^\top$, $b^* = 2.0588$; in both cases the predictions on the test set are identical to those in part (b).

Question 2: SVM with kernels on the attached dataset

Data loading

The data were loaded exactly with the suggested snippet:

```
import pandas as pd
train = pd.read_csv('path/to/training.csv', encoding='utf-8-sig')
test = pd.read_csv('path/to/test.csv', encoding='utf-8-sig')
X_train, y_train = train.iloc[:, :-1].to_numpy().astype(float), train.iloc[:, -1].
    to_numpy().astype(float)
X_test, y_test = test.iloc[:, :-1].to_numpy().astype(float), test.iloc[:, -1].to_numpy().
    astype(float)
```

Models

Model 1: linear kernel (soft-margin, $C = 1.0$). This is the baseline given in the assignment. The linear kernel $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$ finds a straight-line decision boundary; with $C = 1.0$ it tolerates some margin violations. It achieves only $\approx 51.02\%$ test accuracy — the data are not linearly separable.

Model 2: RBF (Gaussian) kernel (soft-margin, $C = 1.0$, $\gamma = 2$). The radial basis function kernel $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\big(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\big)$ maps the data into an infinite-dimensional feature space, so a non-linear (roughly circular) decision boundary can be found. With $C = 1.0$ and $\gamma = 2$ the model has 23 support vectors and reaches **91.84% test accuracy**, well above the required 85%.

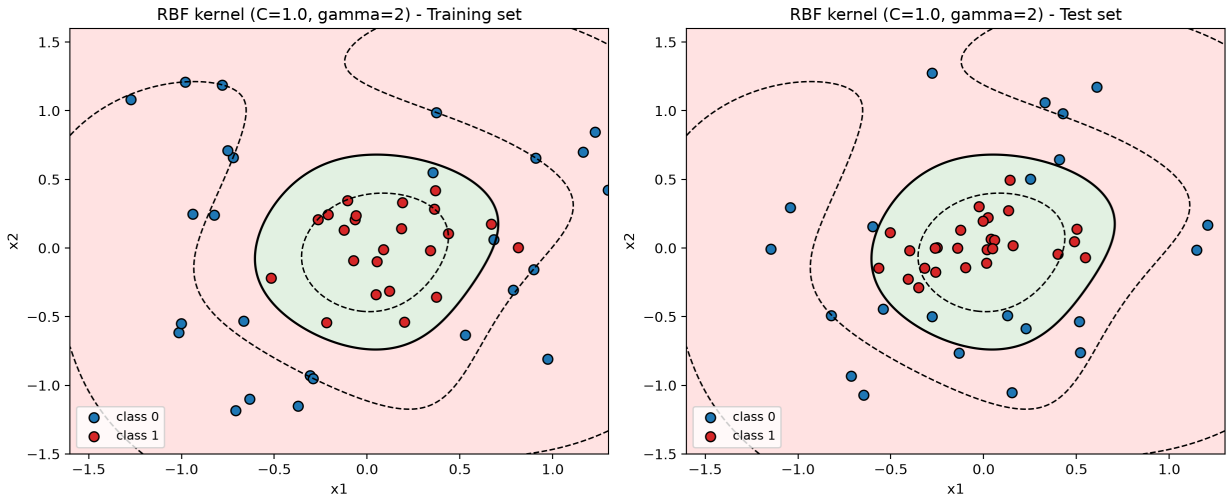
I also tried polynomial ($\approx 67.3\%$) and sigmoid ($\approx 46.9\%$) kernels, and a grid of C values (0.1, 0.5, 1, 2, 5, 10, 50) and of γ (0.1–10) for RBF; the best accuracy 91.84% was reached by several RBF settings, e.g. $C = 1$, $\gamma = 2$ (23 SVs) and $C = 2$, $\gamma = 1$ (19 SVs), so $C = 1.0$, $\gamma = 2$ is reported.

Plots

Decision boundary (solid line) and margins (dashed lines) on both the training and test sets:

Code

The complete script (data loading, both models, and the plotting routine that draws the decision boundary as a solid line and the margins as dashed lines) is shown in the screenshot below (filename



RBF SVM ($C = 1.0$, $\gamma = 2$): decision boundary (solid) and margins (dashed) shown on the training set (left) and test set (right). Test accuracy = 91.84%.

`q2_svm.py`):

Question 3: K-means and DBSCAN

Points: $a(1,2)$, $b(2,3)$, $c(2,4)$, $d(6,5)$, $e(3,1)$, $f(3,3)$, $g(4,4)$, $h(7,6)$, $i(8,8)$, $j(8,9)$, $k(9,4)$, $l(5,9)$, $m(4,7)$, $n(6,8)$, $o(6,4)$.

(a) K-means, first iteration

Initial centres: $\mu_1 = (7,4)$, $\mu_2 = (3,6)$, $\mu_3 = (7,9)$. Using squared Euclidean distance, assign each point to the nearest centre:

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
d_1^2 (to $(7,4)$)	40	26	25	2	25	17	9	4	17	26	4	29	18	17	1
d_2^2 (to $(3,6)$)	20	10	5	10	25	9	5	16	29	34	40	13	2	13	13
d_3^2 (to $(7,9)$)	85	61	50	17	80	52	34	9	2	1	29	4	13	2	26
cluster	2	2	2	1	1	2	2	1	3	3	1	3	2	3	1

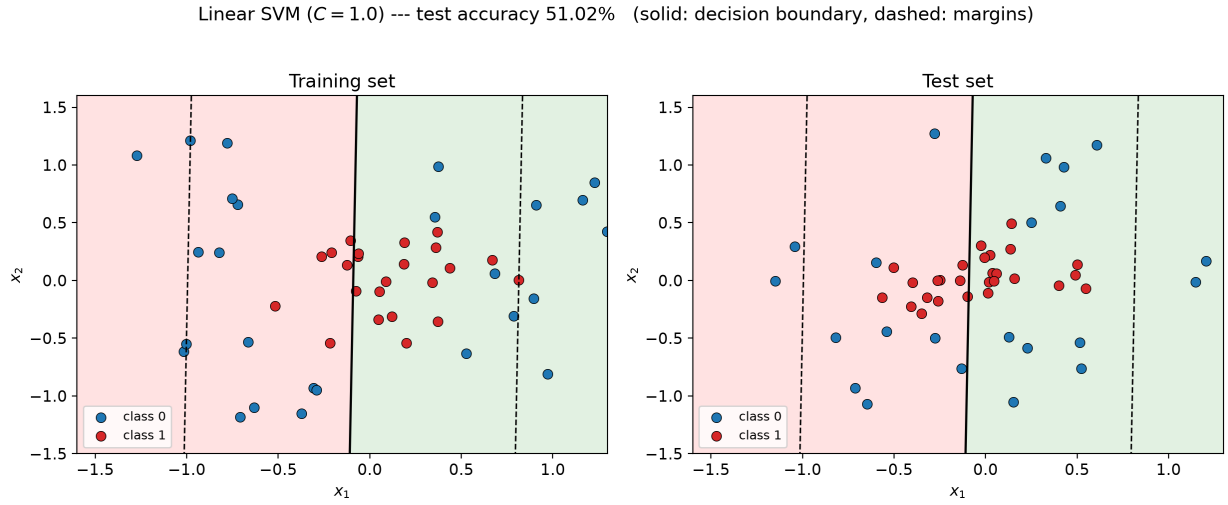
(Ties: e is equally distant from centres 1 and 2 — assigned to cluster 1 here; the tie can be broken arbitrarily without changing the following analysis.)

Updated centres after the first iteration:

$$\mu_1^{(1)} = \text{mean}\{d,e,h,k,o\} = \left(\tfrac{6+3+7+9+6}{5},\ \tfrac{5+1+6+4+4}{5}\right) = \boxed{(6.2,\ 4)},$$

$$\mu_2^{(1)} = \text{mean}\{a,b,c,f,g,m\} = \left(\tfrac{1+2+2+3+4+4}{6},\ \tfrac{2+3+4+3+4+7}{6}\right) = \boxed{(2.6667,\ 3.8333)},$$

$$\mu_3^{(1)} = \text{mean}\{i,j,l,n\} = \left(\tfrac{8+8+5+6}{4},\ \tfrac{8+9+9+8}{4}\right) = \boxed{(6.75,\ 8.5)}.$$



Linear SVM ($C = 1.0$): decision boundary (solid) and margins (dashed). Test accuracy = 51.02%, matching the example in the assignment.

(b) DBSCAN: $\epsilon = 2$, minpts $= 3$

ϵ-neighbourhoods (including the point itself, Euclidean distance):

point	N_ϵ	point	N_ϵ
a	$\{a, b\}$	i	$\{i, j, n\}$
b	$\{a, b, c, f\}$	j	$\{i, j\}$
c	$\{b, c, f, g\}$	k	$\{k\}$
d	$\{d, h, o\}$	l	$\{l, n\}$
e	$\{e, f\}$	m	$\{m\}$
f	$\{b, c, e, f, g\}$	n	$\{i, l, n\}$
g	$\{c, f, g, o\}$	o	$\{d, g, o\}$
h	$\{d, h\}$		

- **Core points** ($|N_\epsilon| \geq 3$): $\boxed{b,\ c,\ d,\ f,\ g,\ i,\ n,\ o}$
- **Border points** (not core, but within ϵ of a core point): $\boxed{a,\ e,\ h,\ j,\ l}$
- **Noise points** (neither core nor border): $\boxed{k,\ m}$

(c) Is h density-reachable from b?

Yes. Density-reachability requires a chain of points $p_0, \ldots, p_k$ with $p_0 = b$, $p_k = h$, where each step goes from a core point to a point in its ϵ-neighbourhood. One such chain is

$$\boxed{b \to c \to g \to o \to d \to h}$$

Check: $c \in N_\epsilon(b)$, $g \in N_\epsilon(c)$, $o \in N_\epsilon(g)$, $d \in N_\epsilon(o)$, $h \in N_\epsilon(d)$, and b, c, g, o, d are all core points. Hence h is density-reachable from b.

q2_svm.py - Assignment2 - Visual Studio Code

q2_svm.py

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC

# ---------- load data (snippet from the assignment) ----------
train = pd.read_csv('training.csv', encoding='utf-8-sig')
test  = pd.read_csv('test.csv',  encoding='utf-8-sig', sep=r'[,\t]+', engine='python')
X_train, y_train = train.iloc[:, :-1].to_numpy().astype(float), train.iloc[:, -1].to_numpy().astype(float)
X_test,  y_test  = test.iloc[:, :-1].to_numpy().astype(float),  test.iloc[:, -1].to_numpy().astype(float)

# ---------- Model 1: linear kernel, soft margin ----------
clf_lin = SVC(kernel='linear', C=1.0)
clf_lin.fit(X_train, y_train)
acc_lin = clf_lin.score(X_test, y_test)
print(f'linear kernel, C=1.0:        test accuracy = {acc_lin:.4f}')

# ---------- Model 2: RBF kernel, soft margin ----------
clf_rbf = SVC(kernel='rbf', C=1.0, gamma=2)
clf_rbf.fit(X_train, y_train)
acc_rbf = clf_rbf.score(X_test, y_test)
print(f'RBF kernel, C=1.0, gamma=2:  test accuracy = {acc_rbf:.4f}')
print(f'number of support vectors = {len(clf_rbf.support_)}')

# ---------- 2D plot: decision boundary (solid) + margins (dashed) ----------
def plot_svm(ax, X, y, clf, title):
    x_min, x_max = -1.6, 1.3
    y_min, y_max = -1.5, 1.6
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 400),
                         np.linspace(y_min, y_max, 400))
    Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    ax.contourf(xx, yy, Z, levels=[-10, 0, 10], colors=['#ffd0d0', '#cfe8cf'], alpha=0.6)
    ax.contour(xx, yy, Z, levels=[0],  colors='k', linewidths=1.6, linestyles='solid')   # decision boundary
    ax.contour(xx, yy, Z, levels=[-1, 1], colors='k', linewidths=1.1, linestyles='dashed') # margins
    ax.scatter(X[y == 0, 0], X[y == 0, 1], c='#1f77b4', s=50, label='class 0', edgecolors='k')
    ax.scatter(X[y == 1, 0], X[y == 1, 1], c='#d62728', s=50, label='class 1', edgecolors='k')
    ax.set_xlabel('x1'); ax.set_ylabel('x2'); ax.set_title(title)
    ax.legend(loc='lower left', fontsize=9)

fig, axes = plt.subplots(1, 2, figsize=(12.5, 5.2))
plot_svm(axes[0], X_train, y_train, clf_rbf, 'RBF kernel (C=1.0, gamma=2) - Training set')
plot_svm(axes[1], X_test,  y_test,  clf_rbf, 'RBF kernel (C=1.0, gamma=2) - Test set')
plt.tight_layout()
plt.savefig('q2_rbf.png', dpi=200)
plt.show()
```

Screenshot of the code used for Question 2.

(d) Density-based clusters

The core points form two connected components (via direct density-reachability between core points):

$$\{b, c, d, f, g, o\} \cup \text{border } \{a, e, h\} \qquad \text{and} \qquad \{i, n\} \cup \text{border } \{j, l\}.$$

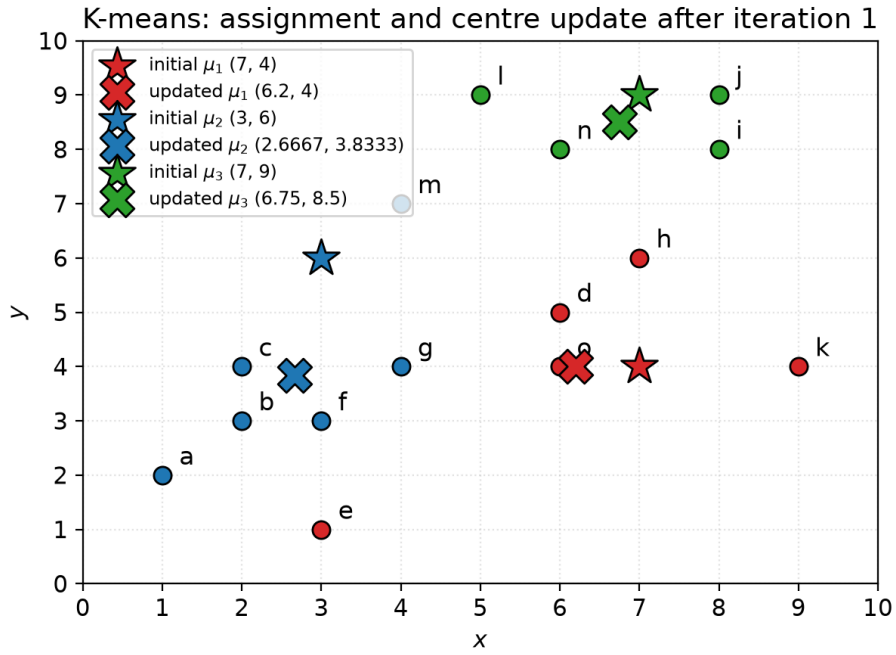
Therefore

- Cluster 1: $\boxed{\{a,\ b,\ c,\ d,\ e,\ f,\ g,\ h,\ o\}}$
- Cluster 2: $\boxed{\{i,\ j,\ l,\ n\}}$

and $k,\ m$ remain noise points (not part of any cluster).

Question 4: Hierarchical clustering (group average)

Distance matrix:



Initial centres (stars) and updated centres after iteration 1 (X marks).

	A	B	C	D	E
A	0	8	6	2	1
B	8	0	7	3	6
C	6	7	0	9	5
D	2	3	9	0	4
E	1	6	5	4	0

Group average distance between clusters U, V: $d(U,V) = \frac{1}{|U||V|} \sum_{u \in U} \sum_{v \in V} d(u,v)$.

Iteration 1. Smallest distance is $d(A,E) = 1$. Merge A and E into AE ($d = 1$). Updated table:

	AE	B	C	D
AE	0	(8+6)/2 = 7	(6+5)/2 = 5.5	(2+4)/2 = 3
B	7	0	7	3
C	5.5	7	0	9
D	3	3	9	0

Iteration 2. Smallest distances: $d(B,D) = 3$ and $d(AE,D) = 3$ (tie; tie-breaking does not affect the final structure). Merge B and D into BD ($d = 3$). Updated table:

$$d(AE,BD) = \frac{8+6+2+4}{4} = \frac{20}{4} = 5, \qquad d(C,BD) = \frac{7+9}{2} = 8, \qquad d(C,AE) = 5.5.$$

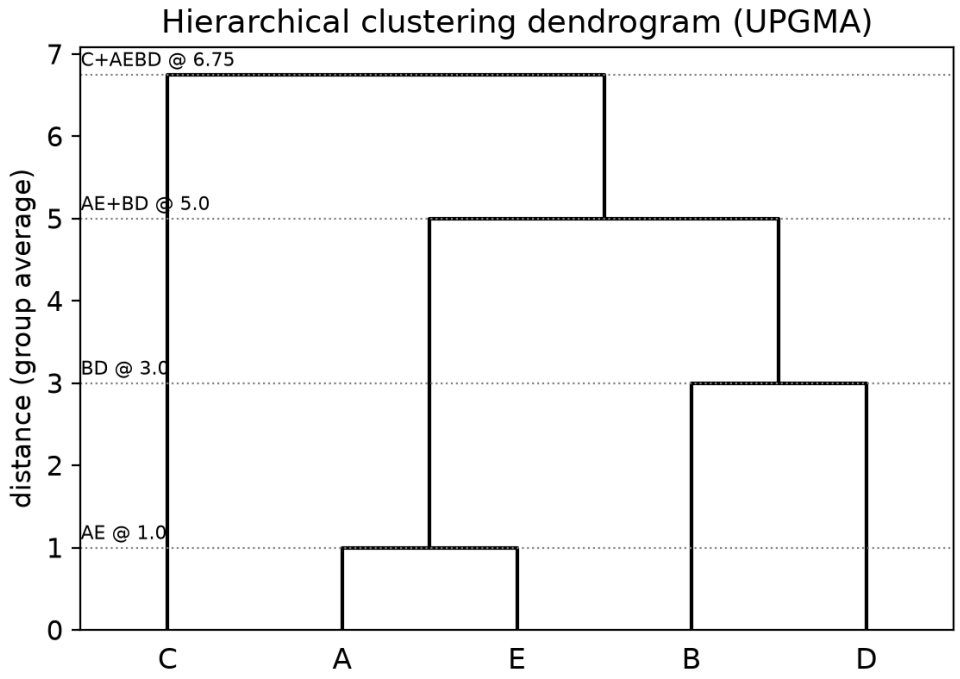
	AE	BD	C
AE	0	5	5.5
BD	5	0	8
C	5.5	8	0

Iteration 3. Smallest distance is $d(AE, BD) = 5$. Merge into $AEBD$ ($d = 5$). Updated table:

$$d(C, AEBD) = \frac{6 + 5 + 7 + 9}{4} = \frac{27}{4} = 6.75.$$

	AEBD	C
AEBD	0	6.75
C	6.75	0

Iteration 4. Merge $AEBD$ and C into the single cluster $ABCDE$ at $d = 6.75$. Done.



Final dendrogram. Merge order: A–E at 1, B–D at 3, AE–BD at 5, then C joins at 6.75.

Summary of the merges (order in which points are merged):

$$\boxed{(A, E) \xrightarrow{1.0} (B, D) \xrightarrow{3.0} (AE, BD) \xrightarrow{5.0} (AEBD, C) \xrightarrow{6.75}}$$